# UNAB-Ambiental

*Informe técnico — Laboratorio 2: Flota IoT Central + generación de datos*

IoT + Cloud + Sistemas Distribuidos · 2026

Integrantes: Andi Lin – U00180153 · Juan Sebastián Buitrago Caballero – U00179631 · Miguel Ángel Solano Díaz – U00177983

## Objetivos

Reutilizar la aplicación y el Device Template del Laboratorio 1, y ampliar la flota a 8 dispositivos con al menos 3 variables cada uno. Generar datos desde un script Python corriendo en una máquina virtual y desde un ESP32 virtual en Wokwi con firmware Arduino. Configurar una Rule por umbral funcional sobre toda la flota y comparar los tres orígenes de datos: simulador nativo, Python-VM y Wokwi.

## Inventario de la flota (8 dispositivos)

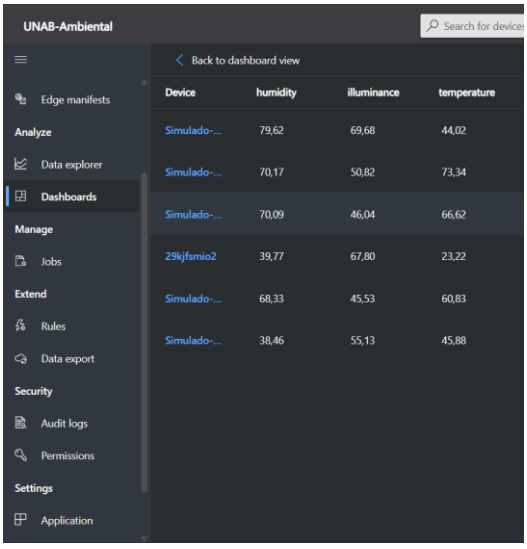
| N | Tipo | Generación de datos | Vars / Estado |
|---|---|---|---|
| 6 | Simulados IoT Central (29kjfsmio2, Simulado-Aula-101 a 105) | Simulador nativo de la aplicación (motor interno de IoT Central) | temperature, humidity, illuminance — Connected |
| 1 | Python en VM (VirtualBox/Ubuntu) | Script sensor_ambiental.py, valores sintéticos aleatorios cada 10s vía azure-iot-device (DPS + SAS) | temperature, humidity, illuminance + comando/property — Connected |
| 1 | ESP32 Arduino en Wokwi | sketch.ino: lectura de DHT22 (temp/hum) + potenciómetro (illuminance), publicación MQTT vía DPS implementado a mano (mbedtls) | temperature, humidity, illuminance + LED (setAlertLed) — Connected |

## Cómo se genera el dato en cada nodo

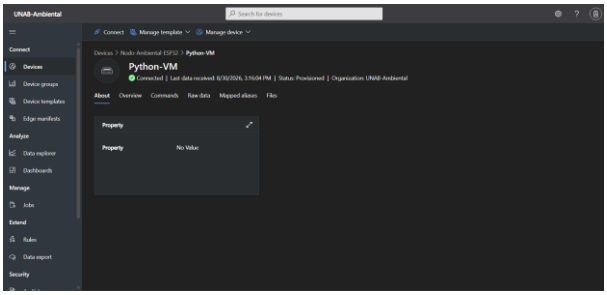
Python-VM: dentro de una máquina virtual Ubuntu (VirtualBox), el script sensor_ambiental.py se aprovisiona vía Device Provisioning Service (DPS) usando ID Scope, Device ID y Primary Key, y publica temperature/humidity/illuminance cada 10 segundos generados con random.uniform dentro de los rangos operativos del escenario (16–32 °C, 35–70 % RH, 100–800 lux). Además escucha comandos remotos (reiniciar, lecturaForzada) y la property writable modoAlerta.

ESP32-Wokwi: el firmware en Arduino lee el sensor DHT22 (temperatura y humedad) y un potenciómetro que simula el sensor de luz (illuminance), y publica los mismos tres valores por MQTT sobre TLS. Como no hay SDK oficial de Azure vigente para Arduino, el aprovisionamiento DPS y los tokens SAS se implementaron manualmente con la librería mbedtls. El comando setAlertLed enciende un LED físico simulado en el circuito.

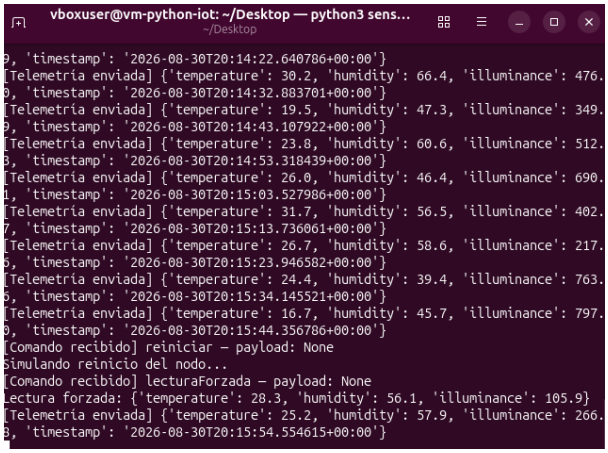
## Capturas clave

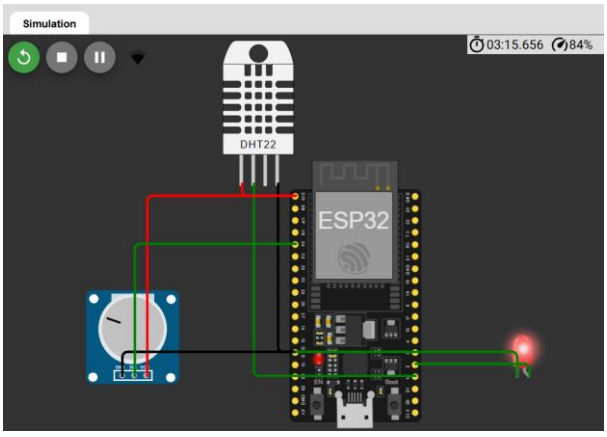


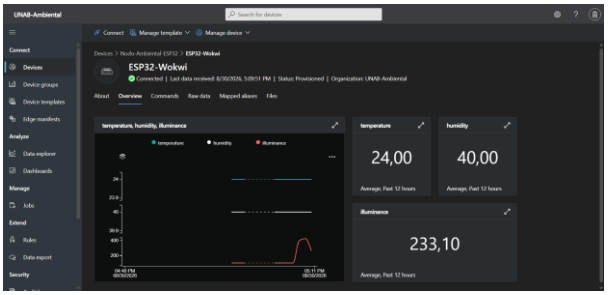
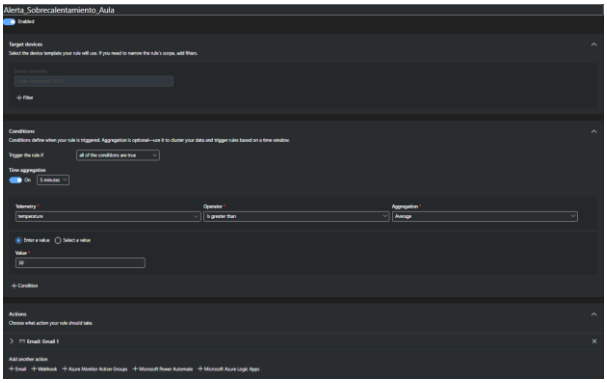

*Dashboard de flota: 6 simulados con Last Known Value*




*Python-VM: Connected y gráficas en IoT Central*



| *Python-VM: terminal enviando telemetría y recibiendo comandos* | *Wokwi: circuito ESP32 + DHT22 + potenciómetro + LED encendido tras 'setAlertLed'* |
|---|---|
|  *ESP32-Wokwi: Connected y telemetría en IoT Central* |  *Rule Alerta_Sobrecalentamiento_Aula (aplica a toda la flota)* |

## Regla por umbral

Se reutilizó la Rule Alerta_Sobrecalentamiento_Aula del Laboratorio 1 (condición temperature > 30 °C, Average, ventana de 5 minutos, acción Email), la cual aplica automáticamente a los 8 dispositivos por estar definida sobre el mismo Device Template. Se forzó el disparo y se confirmó la notificación recibida (evidencia en el repositorio del Lab 1, reutilizada aquí por ser la misma regla y el mismo template).

## Tabla comparativa: Simulador vs Python-VM vs Wokwi

| Criterio | Simulador nativo | Python-VM | Wokwi (ESP32) |
|---|---|---|---|
| Origen del dato | Motor interno de IoT Central (aleatorio, sin código) | Generado por script (random.uniform en rangos definidos) | Leído de sensores virtuales; requiere interacción manual para variar |
| Latencia percibida | Inmediata (pipeline interno) | Baja y estable (salida directa a Internet de la VM) | Algo mayor y menos predecible (pasa por Wokwi IoT Gateway) |
| Facilidad de debug | Nula (caja negra, sin logs propios) | Alta (logs completos por consola de cada envío/comando) | Alta vía consola serial, mezclada con logs de bajo nivel del boot del ESP32 |
| Complejidad de implementación | Ninguna (todo por UI) | Media (SDK oficial azure-iot-device, venv, variables de entorno) | Alta: sin SDK vigente para Arduino, SAS/DPS implementado a mano con mbedtls |
| Fallos de red/gateway | Ninguno (no depende de infraestructura propia) | Ninguno tras resolver cuota 0 de VM en Azure for Students | Desconexión/reconexión MQTT intermitente al inicio; resuelta subiendo keepalive de 15s a 60s |

## Observaciones y limitaciones

La cuenta Azure for Students trae cuota 0 por defecto en casi todas las familias de VM, incluso en regiones no habituales — fue necesario solicitar aumento de cuota antes de poder crear cualquier máquina virtual en la nube; finalmente se optó por una VM local (VirtualBox) para evitar la demora de aprobación.

Wokwi, al depender de una red virtual compartida (Wokwi-GUEST) y un gateway público hacia Azure, mostró menor estabilidad de conexión que la VM local, evidenciando que la capa de telecomunicaciones es la parte más frágil de un despliegue IoT real, algo que el simulador nativo de IoT Central nunca expone porque no pasa por una red física real.

## Repositorio GitHub

https://github.com/AndiLinUnab/unab-ambiental-lab2.git